

Observability & Monitoring (Prometheus, PromQL, Grafana)

Category: **Telemetry, Metrics & Production Diagnostics** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Observability & Monitoring (Prometheus, PromQL, Grafana)**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- **The Four Golden Signals of Monitoring:** Google SRE methodology focusing on Latency (time to serve a request), Traffic (demand/request volume), Errors (rate of failing requests), and Saturation (how full service resources are).

Production Application: Forms the standard top-level dashboard layout for all enterprise services, allowing on-call engineers to diagnose 95% of outages in seconds.

- **PromQL Vector Types & Rate Mechanics:** Instant Vectors (single sample per series at a point in time), Range Vectors (range of samples over time, e.g. `[5m]`), and Scalar values. `rate()` calculates per-second average rate of increase for monotonically increasing counters, handling counter resets automatically.

Production Application: Never use `rate()` on Gauges (like memory or queue length); use `rate()` exclusively on Counters (like `http_requests_total`).

- **High Cardinality Pitfalls & Metric Relabeling:** Cardinality is the total number of unique time series generated by the Cartesian product of label keys and values. Adding dynamic labels (e.g. user IDs, email addresses, order IDs) creates millions of series, crashing Prometheus with out-of-memory errors.

Production Application: Use `metric_relabel_configs` to drop or sanitize high-cardinality labels before ingestion, protecting Prometheus TSDB storage.

- **Multi-Window Multi-Burn-Rate Alerting:** Advanced alerting methodology from the Google SRE Book that alerts based on how fast an application is consuming its Error Budget across multiple time windows (e.g. 1 hour vs 6 hours).

Production Application: Eliminates alert fatigue by alerting PagerDuty only on urgent severe burn rates while sending slow chronic burn rates to email/ticket queues.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
<pre> histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket{service='payment'}[5m]))) histogram_quantile(,) </pre>	Calculate the 99th percentile (P99) request latency over a rolling 5-minute window for the payment service. Options: [{flag: '0.99', 'description': 'Calculate 99th percentile (99% of requests completed within this time).'}, {flag: 'sum by (le)', 'description': 'Mandatory PromQL aggregation clause across histogram bucket boundaries (le).'}, {flag: '[5m]', 'description': 'Range vector window over which counter rates are calculated.'}]	Embed into Grafana dashboards and Alertmanager rules to detect latency spikes before customers churn. Pitfall: Omitting `le` from the `sum by (...)` aggregation, causing Prometheus to return `NaN` or incorrect calculations.
<pre> sum(rate(http_requests_total{status=~'5..'}[5m])) / sum(rate(http_requests_total[5m])) * 100 sum(rate({})) / sum(rate({})) * 100 </pre>	Calculate the real-time percentage error rate of HTTP 5xx responses relative to total incoming traffic. Options: [{flag: "status=~'5..'", 'description': 'Regex filter matching HTTP status codes 500 through 599.'}, {flag: 'sum(...)', 'description': 'Aggregate across all pods, hosts, and endpoints to compute cluster-wide ratio.'}]	Monitored by automated deployment systems (Argo Rollouts, Spinnaker) to automatically abort deployments if error rate exceeds 1%. Pitfall: Dividing by zero when there is no incoming traffic, causing the expression to return no data.
<pre> 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode='idle'}[5m])) * 100) 100 - (avg by () (rate(node_cpu_seconds_total{mode='idle'}[5m])) * 100) </pre>	Calculate the real-time CPU utilization percentage per host instance over a rolling 5-minute interval. Options: [{flag: "mode='idle'", 'description': 'Isolate time CPU spent idle; subtracting from 100 yields active CPU utilization.'}, {flag: 'avg by (instance)', 'description': 'Average across all CPU cores per host instance.'}]	Used to detect host CPU bottlenecks and configure automated cluster node autoscaling. Pitfall: Calculating CPU usage by summing non-idle modes rather than subtracting idle, which ignores steal and guest time.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
strace -p -f -e trace=network,file -s 256 strace [options] -p	Attach to a running Linux process and trace all network and file system system calls with expanded string sizes in real time. Options: [{ 'flag': '-p ', 'description': 'Process ID of the target process to attach to.' }, { 'flag': '-f', 'description': 'Trace child threads and forks created by the process.' }, { 'flag': '-e trace=network,file', 'description': 'Filter syscalls strictly to network (socket, connect, sendto) and file operations.' }, { 'flag': '-s 256', 'description': 'Display up to 256 characters for each string argument.' }]	Run during severe incidents to identify whether a process is stuck waiting on a hung network socket, file lock, or database connect. Pitfall: Running `strace` on high-throughput production processes without filtering, which causes severe performance overhead (up to 50x slowdown).
lsof -i :8080 -sTCP:LISTEN lsof [options]	List all processes holding open listening sockets on TCP port 8080 with process names and PIDs. Options: [{ 'flag': '-i :8080', 'description': 'Filter network sockets matching port 8080.' }, { 'flag': '-sTCP:LISTEN', 'description': 'Restrict output to sockets in the TCP LISTEN state.' }]	Execute during deployment failures where an application crashes with 'Address already in use'. Pitfall: Running lsof without root/sudo, which silently hides processes owned by other system users.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.